PROJECT 29 · 22AIE203 DATA STRUCTURES AND ALGORITHMS 2

Intelligent Emergency Resource Allocation & Ambulance Dispatch System

Viva Guide — Project Walkthrough and Algorithms Explained with Code

Orthogonal Range Trees · Fractional Cascading · Point Location · Java

TEAM

SHARATH SACHIN S	CH.SC.U4AIE25071
NITESH RAM RE	CH.SC.U4AIE25075
SIVAGURU R M	CH.SC.U4AIE25044
RAHUL BHARADWAJ	CH.SC.U4AIE25039

GUIDE

BHARATHI MOHAN

Department of AIE · Amrita Vishwa Vidyapeetham · Academic Year 2026–27

Contents

Part A — Presenting the Project 3

1. The Opening (first 60 seconds) 3
2. Why the Naive Approach Fails 3
3. The Three Data Structures (core of the viva) 4
4. The System Layer 5
5. The Live Demo (run it, don't describe it) 6
6. Correctness Evidence (lead with this, don't wait to be asked) 6
7. Limitations (say these before he finds them) 6
8. Questions He Will Very Likely Ask 7

Part B — The Algorithms, Explained with Code 8

Map of the Algorithms 8
Algorithm 1 — Building the Range Tree 9
Algorithm 2 — Merge by y and the Fractional-Cascading Pointers 10
Algorithm 3 — Binary Search (lower bound) 12
Algorithm 4 — Range Query with Fractional Cascading (the main algorithm) 12
Algorithm 5 — Nearest Neighbour (growing window) 15
Algorithm 6 — Radius Search 16
Algorithm 7 — Point Location (slab decomposition) 17
Algorithm 8 — Allocation by Severity 19
Algorithm 9 — Redistribution 20
Algorithm 10 — Coverage Map 22
Algorithm 11 — Checking Against Brute Force 22
Complexity Summary 23

Part A — Presenting the Project

1. The Opening (first 60 seconds)

> "Sir, our project is **Project 29 — Intelligent Emergency Resource Allocation and Ambulance Dispatch System**. The problem is that in a real city, a 108 control room is holding hundreds of ambulances, fire engines, police units and hospitals on a map, and it has to answer the same four questions over and over, under time pressure: *which units are free in this district? which ambulance is nearest to this accident? which hospitals within 5 km have an ICU bed? and after I dispatch, where am I now blind?*
>
> If you answer those by scanning the whole resource list, every single query costs **O(n)**. Our claim is that this is fundamentally a **computational geometry** problem, not a database problem. So we pre-process the resource positions into geometric data structures and answer the same questions in **O(log n + k)** time — logarithmic in the number of resources, plus the size of the answer.
>
> We implemented three structures from scratch in Java, with no libraries: a **2-D orthogonal range tree**, **fractional cascading** on top of it, and **point location by slab decomposition**. Then we built a realistic dispatch layer on top of them."

That framing matters: it tells the professor immediately that the DSA content is the *substance*, not decoration on a CRUD app.

2. Why the Naive Approach Fails

Make the cost explicit before you show the fix:

Query	Naive	Ours
Free ambulances in a district	O(n) scan	**O(log n + k)**
Nearest free ambulance	O(n) scan	**O(log n + k)** per window
ICU hospitals within 5 km	O(n) scan	**O(log n + k)**
Which sector owns this call?	O(z) over zones	**O(log n)**

> "The key insight is that a dispatch system is **query-heavy and update-light**. Resource *positions* barely change; only their *availability* flips. That asymmetry is exactly what justifies paying an O(n log n) preprocessing cost once to make every subsequent query logarithmic."

That sentence is the single best thing you can say — it is the *engineering justification* for choosing a range tree at all.

3. The Three Data Structures (core of the viva)

3.1 Orthogonal Range Tree — `RangeTree2D.java`

Explain it as two layers:

- **Primary tree**: a balanced BST built on the **x-coordinate**. All 16 resources sit in the leaves. Each internal node stores `splitX` = the largest x in its left subtree, used purely for routing.
- **Associated structure**: every node also stores an array `ys[]` of *all points in its subtree, sorted by y*.

A query for the rectangle `[x1,x2] × [y1,y2]` works like this:

1. Walk down until the path to x1 and the path to x2 diverge — that node is the **split node**.
2. From there, walk the left spine following x1 and the right spine following x2.
3. Along the way you hang off **O(log n) "canonical nodes"** — subtrees whose entire x-range is inside `[x1,x2]`.
4. At each canonical node, binary search its y-array for y1 and report forward while `y ≤ y2`.

Cost: O(log n) canonical nodes × O(log n) binary search each + k output = **$O(\log^2 n + k)$**.

Build is O(n log n) because each level merges y-arrays in linear time. Space is **O(n log n)** — and you should volunteer why: *every one of the log n levels stores all n points in its y-arrays.*

3.2 Fractional Cascading — the part that earns the marks

This is your headline contribution, so slow down here.

> "Notice the waste: at every canonical node we binary search for the **same** threshold y1, from scratch, independently. Fractional cascading removes that redundancy."

The mechanism, in `mergeByY()` (`RangeTree2D.java:115`):

- When merging the two children's y-arrays into the parent's, we also record, for every entry `i` of the parent, **two pointers**: `leftPtr[i]` = the first index in the *left* child's array with y ≥ `parent.ys[i].y`, and `rightPtr[i]` likewise for the right child.
- These are computed by a single backward sweep with two moving fingers — **linear time**, so build stays O(n log n).

Now the query does **one** binary search, at the root only. Every step down the tree just follows a pointer in O(1) — `pos = v.leftPtr[pos]`. The y-threshold is carried down the whole traversal for free.

Cost drops to the optimal O(log n + k).

Be ready for "prove the pointer is correct." The professor will ask this. The answer:

> "A child's y-array is a **subsequence** of its parent's. So if `i` is the first parent index with y ≥ y1, then the first child element with y ≥ `parent.ys[i].y` is *exactly* the first child element with y ≥ y1 — because no child element can lie strictly between y1 and `parent.ys[i].y`. If one did, it would also be in the parent's array and would have come before index `i`, contradicting that `i` was first. So the stored pointer lands on the correct position without re-searching."

Also mention the **sentinel**: the pointer arrays have length `n+1`, so a position that has run off the end still maps validly to a child.

3.3 Point Location (slab decomposition) — `PointLocation.java`

> "Separately from *which unit*, the control room needs to know *which sector owns this call*. That's the classical **planar point location** problem."

- Every vertical edge of every zone rectangle induces a vertical line; consecutive lines bound vertical **slabs**.
- Within one slab, the zones appear as a set of **disjoint y-intervals**, stored sorted by lower edge.
- A query binary searches x to pick the slab, then binary searches y within it → **O(log n)**.
- Honest trade-off to state yourself: preprocessing is **$O(n^2)$ space worst case**, because a rectangle can appear in many slabs.

3.4 Nearest Neighbour — and why it is provably correct

This is the second-most-likely deep question. It is in `nearest()` (`RangeTree2D.java:276`):

1. Start with a 1 km square window around the incident; run a range query filtered by the predicate (e.g. "is an available ambulance").
2. **Double the window** until you get a hit, or until the window covers the whole map.
3. Take the best Euclidean distance `d` among those hits.
4. **Do one final exact range query over the box of half-side `d`**, and take the true minimum.

> "Step 4 is what makes it provably correct, not just a heuristic. The square window is a *superset* of the circle of radius d — any point closer than `d` in Euclidean distance **must** lie inside that box. So re-querying the box of half-side `d` cannot miss the true nearest neighbour."

The same logic gives the radius query: bounding-box range query, then exact Euclidean filter (`withinRadius()`).

4. The System Layer

`DispatchSystem.java` is a façade holding both indexes and exposing the six queries from the brief.

- **Severity policy** — `Severity.java` encodes it declaratively: `CRITICAL` → 2 ambulances + ICU required; `HIGH` → 1 + ICU; `MODERATE` / `LOW` → 1, no ICU.
- **Incident type** — `IncidentType.java` decides support units: a `ROAD_ACCIDENT` pulls a fire truck *and* police; a `CARDIAC_ARREST` pulls neither.
- **Allocation** (`allocate()`, `DispatchSystem.java:135`) composes them: locate the zone, pull the N nearest ambulances, mark each busy, reserve an ICU bed, add fire/police as the type demands, and record anything it *couldn't* satisfy in an `unmet` list. That last bit matters — a real dispatcher needs to know it is short, not get a silent failure.
- **Redistribution** (`redistributeCoverage()`, `DispatchSystem.java:196`): any zone left with zero available ambulances pulls the nearest idle ambulance from a zone with a *surplus* — with a guard that refuses to strip a donor zone down to zero. Relocation physically moves the unit and **rebuilds the index**.

> "Note the design decision: **availability is a mutable flag, so dispatch and return-to-service need no rebuild at all**. Only *relocation* changes a coordinate, and only that triggers the O(n log n) rebuild."

5. The Live Demo (run it, don't describe it)

> **Practical warning before the viva:** this machine has a Java 8 JRE first on PATH but the classes are compiled for Java 17 — a bare `java -cp out emergency.Main` throws `UnsupportedClassVersionError`. Use the script, which auto-detects the JDK 17 install.

```
run.bat
```

Then walk them through the output, pointing at each structure:

- **Query 1** — South-West region `[0..15] × [0..15]` returns A1 and A2. "*That's the orthogonal range search.*"
- **Query 2** — accident at (18, 23) → **A5 at 4.47 km**. "*Nearest neighbour via expanding window.*"
- **Query 3** — around (7, 11), radius 5 km → **City-General, 3.16 km, 6 ICU beds**. "*Range query plus exact circle filter plus ICU predicate.*"
- **Query 4** — critical road accident at (19, 24): **point location routes it to North-East**, then 2 ambulances (A5, A4) + ICU hospital H2 + fire engine F2 + patrol P1.
- **Query 5** — the best demo moment. Two back-to-back calls consume A7 and A8, so South-East shows **0 ambulances, response 13.29 km, flagged UNDER-COVERED**. Redistribution relocates A6 from North-East to the sector centre and the response drops to **0.00 km**.

```
run.bat test
```

All **9 checks pass**.

6. Correctness Evidence (lead with this, don't wait to be asked)

`Verification.java` is a hand-written suite with no external framework. The important design point:

> "We don't test the structure against itself. Every spatial result is checked against an **independent brute-force O(n) reference** over hundreds of randomised point sets — about **6,000 range queries, 3,000 nearest-neighbour queries and 2,000 radius queries**, each required to match exactly."

Plus edge cases that are exactly where geometric code breaks: empty tree, single-point tree, and **duplicate coordinates** (handled by the half-open routing rule, so no coordinate perturbation is needed). Plus three behavioural checks on point location, allocation and redistribution.

```
RESULT: 9 passed, 0 failed
ALL CHECKS PASSED
```

7. Limitations (say these before he finds them)

Volunteering weaknesses reads as maturity; being caught on them reads as the opposite.

1. **Euclidean distance, not road distance.** Straight-line km is a proxy. A real deployment needs a road network with shortest-path / travel-time weighting — an ambulance 2 km away across a river is not 2 km away.

2. **Relocation rebuilds the entire index** in O(n log n). A dynamic range tree or a kd-tree supporting deletion would update incrementally.
3. **Point location is O(n^2) space worst case.** A trapezoidal map or persistent search tree achieves O(n) space with the same O(log n) query.
4. **Some system-layer helpers are still linear.** `returnToService` (`DispatchSystem.java:184`) and `nearestSurplusAmbulance` (`DispatchSystem.java:217`) scan the resource list. The *spatial* queries are logarithmic; these bookkeeping paths could be indexed by a hash map. Worth conceding directly if asked — it is an honest, bounded gap, not a flaw in the geometry.

8. Questions He Will Very Likely Ask

"Why a range tree and not a kd-tree or quadtree?"

> A kd-tree uses O(n) space but its range query is O(√n + k) — worse asymptotically. A range tree pays O(n log n) space to buy O($\log^2$ n + k), and fractional cascading takes it to the optimal O(log n + k). Since our workload is query-heavy and memory is cheap relative to response time, we deliberately traded space for query speed. A quadtree has no worst-case guarantee at all — it degrades badly under clustered data, and emergency resources *are* clustered.

"Where exactly does the second log go?"

> It's the per-canonical-node binary search. With cascading we binary search once at the root and follow O(1) pointers thereafter, so the O(log n) canonical nodes cost O(1) each to enter instead of O(log n).

"What is k?"

> The number of points reported. It's output-sensitive — you can never beat O(k), since you must at minimum write down the answer. That's why O(log n + k) is called *optimal*.

"How do you handle two ambulances at the same coordinates?"

> The standard half-open routing rule: `splitX` is the max x in the left subtree, x1 is inclusive-left and x2 is strict-left, and leaves are explicitly tested for containment. We test this specifically — there's a check that four points at identical coordinates are all reported.

"Is the nearest-neighbour search a heuristic?"

> No. The expanding window only *finds a candidate distance d*; correctness comes from the second exact pass over the box of half-side d, which provably contains every point closer than d.

"What would you do next?"

> Road-network travel time instead of Euclidean, incremental index updates, multi-incident batching, and predictive pre-positioning — moving ambulances *before* calls arrive, based on historical demand.

The sentence to close on

> "The point of the project isn't the dispatch simulation — it's that three classical computational-geometry structures turn an O(n)-per-query operational problem into an O(log n + k) one, and we verified that claim against brute force rather than just asserting it."

Part B — The Algorithms, Explained with Code

These are the algorithms in the project, in the order the data flows through them. For each one: the idea, the actual code, what the code is doing, and a worked example. The examples were not worked out by hand — the real tree's internal arrays were dumped from the compiled classes and the real queries were run, so every number matches what the program produces.

To keep the traces readable, the range tree examples use only the **8 ambulances**. The full program builds the same kind of tree over all 16 resources.

Map of the Algorithms

#	Algorithm	File	Time
1	Build the range tree	`RangeTree2D.java:67`	O(n log n)
2	Merge by y + fractional-cascading pointers	`RangeTree2D.java:115`	O(n) per node
3	Binary search (lower bound)	`RangeTree2D.java:235`	O(log n)
4	**Range query with fractional cascading**	`RangeTree2D.java:156`	**O(log n + k)**
5	Nearest neighbour (growing window)	`RangeTree2D.java:276`	O(log n + k) per window
6	Radius search	`RangeTree2D.java:259`	O(log n + k)
7	Point location (slab decomposition)	`PointLocation.java`	O(log n) query
8	Allocation by severity	`DispatchSystem.java:135`	O(log n + k) per unit
9	Redistribution	`DispatchSystem.java:196`	O(n log n) per move
10	Coverage map	`DispatchSystem.java:254`	O(z · (log n + k))
11	Checking against brute force	`Verification.java:61`	test only

Algorithm 1 — Building the Range Tree

Idea

Build a balanced binary search tree on **x**, with every point in a leaf. Each node also keeps a copy of all points in its subtree, **sorted by y**.

Code

```
public RangeTree2D(List<Point<T>> points) {
    ...
    Point<T>[] byX = newArray(points.size());
    points.toArray(byX);
    Arrays.sort(byX, new Comparator<Point<T>>() {        // step 1: sort by x
        public int compare(Point<T> a, Point<T> b) {
            return Double.compare(a.x(), b.x());
        }
    });
    ... // compute bounding box minX, maxX, minY, maxY
    root = build(byX, 0, byX.length - 1);                // step 2: recursive build
}

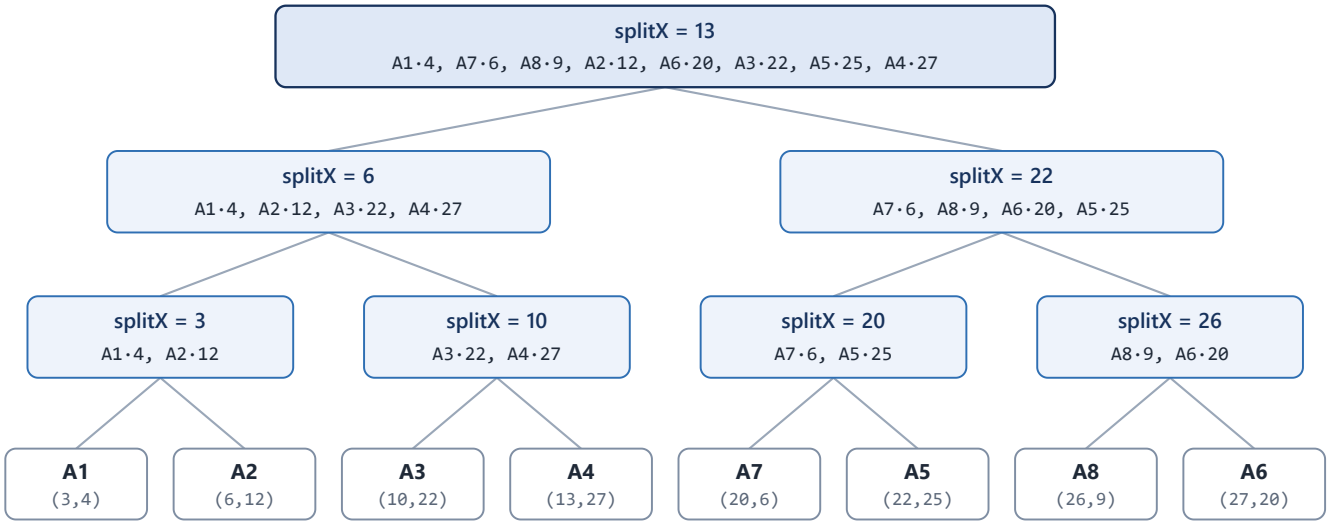
private Node<T> build(Point<T>[] byX, int lo, int hi) {
    Node<T> node = new Node<T>();
    if (lo == hi) {                       // base case: one point -> leaf
        node.isLeaf = true;
        node.point = byX[lo];
        node.ys = newArray(1);
        node.ys[0] = byX[lo];
        return node;
    }
    int mid = (lo + hi) >>> 1;
    node.splitX = byX[mid].x();           // largest x in the left subtree
    node.left  = build(byX, lo, mid);
    node.right = build(byX, mid + 1, hi);
    mergeByY(node);                       // build ys[] + cascade pointers
    return node;
}
```

What the code does

1. **Sort by x once**, in O(n log n).
2. **Split at the middle index** every time. That keeps the tree balanced, so its height is ⌈log n⌉.
3. **`splitX` is the largest x in the left subtree.** Queries use it to choose a direction: if x ≤ splitX, the point is on the left.
4. **`>>> 1`** is an unsigned shift. It computes the midpoint without integer overflow — the classic binary-search bug.
5. **Leaves** store one point, and their **`ys`** array has length 1.
6. **Internal nodes** don't sort their **`ys`** from scratch. They **merge the children's arrays**, which are already sorted.

> **Viva point:** the build is **merge sort on y running alongside the BST build on x**. Every level of recursion does O(n) total merging over log n levels, so the whole build is O(n log n). You never pay $O(n \log^2 n)$ for re-sorting at each node.

The actual tree built from the 8 ambulances



Leaves hold the points in x order. Every internal node stores `splitX` (largest x in its left subtree) and its subtree's points sorted by y. `A7·6` means ambulance A7, whose y is 6.

Space: O(n log n). Every level of the tree holds all n points across its `ys` arrays, and there are log n levels.

Algorithm 2 — Merge by y and the Fractional-Cascading Pointers

This is the preprocessing that makes the fast query possible.

Idea

While merging the two children's y-sorted arrays into the parent's, record **two pointers** for each parent entry:

- `leftPtr[i]` = the first index in the **left** child's array with y ≥ `parent.ys[i].y`
- `rightPtr[i]` = the same thing for the **right** child

Code

```
private void mergeByY(Node<T> v) {
    Point<T>[] L = v.left.ys;
    Point<T>[] R = v.right.ys;
    int n = L.length + R.length;
    Point<T>[] merged = newArray(n);

    // (a) Standard merge-sort merge on y
    int i = 0, j = 0, k = 0;
    while (i < L.length && j < R.length) {
        merged[k++] = (L[i].y() <= R[j].y()) ? L[i++] : R[j++];
    }
    while (i < L.length) merged[k++] = L[i++];
    while (j < R.length) merged[k++] = R[j++];
    v.ys = merged;

    // (b) Cascade pointers, computed by one backward sweep
    int[] lp = new int[n + 1];
    int[] rp = new int[n + 1];
    lp[n] = L.length;              // sentinel: "past the end"
    rp[n] = R.length;
```

```
        int jl = L.length, jr = R.length;
        for (int t = n - 1; t >= 0; t--) {
            double yv = merged[t].y();
            while (jl - 1 >= 0 && L[jl - 1].y() >= yv) jl--;
            lp[t] = jl;
            while (jr - 1 >= 0 && R[jr - 1].y() >= yv) jr--;
            rp[t] = jr;
        }
        v.leftPtr = lp;
        v.rightPtr = rp;
    }
```

What the code does

Part (a) is the textbook merge. `<=` makes it **stable**: when two y-values tie, the left child's point goes first.

Part (b) walks the merged array **from right to left** with two fingers, `jl` and `jr`:

- For each parent entry with value `yv`, it pulls `jl` left as long as the left child's element just before it is still ≥ `yv`. When it stops, `jl` is the first left-child index with y ≥ `yv`.
- `jr` does the same for the right child.
- **Why this is linear:** the fingers only ever move left. `jl` can move at most `L.length` times and `jr` at most `R.length` times over the whole loop. The inner `while` loops therefore do O(n) work in total, not O(n) per iteration.

The sentinel at index `n` matters. If y1 is larger than every y in a node, the search returns position `n`. Then `leftPtr[n]` must still be a valid slot, and it points past the end of the child's array.

Trace: the root node

The merge inputs are L = `[A1·4, A2·12, A3·22, A4·27]` and R = `[A7·6, A8·9, A6·20, A5·25]`.

t	0	1	2	3	4	5	6	7	8 (sentinel)
`ys[t]`	A1·4	A7·6	A8·9	A2·12	A6·20	A3·22	A5·25	A4·27	—
`leftPtr`	0	1	1	1	2	2	3	3	4
`rightPtr`	0	0	1	2	2	3	3	4	4

Read column 3: parent entry `A2·12`. `leftPtr[3] = 1` means the first left-child value ≥ 12 is `L[1] = A2·12`. `rightPtr[3] = 2` means the first right-child value ≥ 12 is `R[2] = A6·20`. (These pointer values come straight from the running program.)

Why the pointers are correct

> A child's array is a **subsequence** of its parent's. Say `pos` is the first parent index with y ≥ y1. Could a child element have y between y1 and `parent[pos].y`? If it did, it would also be in the parent's array, before `pos`, which contradicts `pos` being first. So "first child element ≥ `parent[pos].y`" is the same as "first child element ≥ y1". The stored pointer lands in exactly the right place without searching again.

Algorithm 3 — Binary Search (lower bound)

Code

```
/** First index i with ys[i].y >= key; returns ys.length if none. */
private int lowerBoundY(Point<T>[] ys, double key) {
    int lo = 0, hi = ys.length;          // search the half-open range [lo, hi)
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        if (ys[mid].y() >= key) hi = mid; // mid might be the answer; keep it
        else lo = mid + 1;                // mid is too small; drop it
    }
    return lo;
}
```

What the code does

- **Invariant:** everything before `lo` is < key, and everything from `hi` onward is ≥ key. When `lo == hi`, that position is the first element ≥ key.
- Starting with `hi = ys.length` (not `length - 1`) is what makes "not found" safely return `ys.length`, which lines up with the sentinel pointer.
- **The query calls this exactly once, at the root.** That single call is the whole benefit of fractional cascading.

Algorithm 4 — Range Query with Fractional Cascading (the main algorithm)

Idea

Report every point inside the rectangle `[x1,x2] × [y1,y2]` in three phases:

1. **Go down to the split node**, where the paths to x1 and x2 separate.
2. **Walk the left spine** following x1. Whenever the path goes left, the right child's whole subtree is inside the x-range, so it is a **canonical node**.
3. **Walk the right spine** following x2, mirrored: whenever the path goes right, the left child is canonical.

For each canonical node, report its `ys` array starting from the **carried-down position** while y ≤ y2.

Code

```
public List<Point<T>> rangeQuery(Rectangle rect) {
    List<Point<T>> out = new ArrayList<Point<T>>();
    if (root == null) return out;
    final double x1 = rect.minX(), x2 = rect.maxX();
    final double y1 = rect.minY(), y2 = rect.maxY();

    // PHASE 1: ONE binary search, then go down to the split node
    Node<T> v = root;
    int pos = lowerBoundY(root.ys, y1);
    while (!v.isLeaf) {
        if (x2 < v.splitX) {              // whole range is to the left
            pos = v.leftPtr[pos];
```

```
            v = v.left;
        } else if (x1 > v.splitX) {       // whole range is to the right
            pos = v.rightPtr[pos];
            v = v.right;
        } else {
            break;                         // paths separate: split node
        }
    }
    if (v.isLeaf) {                        // query narrowed to a single leaf
        addLeafIfInside(v.point, x1, x2, y1, y2, out);
        return out;
    }
    final Node<T> split = v;
    final int posSplit = pos;

    // PHASE 2: left spine follows x1; right children are canonical
    Node<T> lv = split.left;
    int lpos = split.leftPtr[posSplit];
    while (!lv.isLeaf) {
        if (x1 <= lv.splitX) {
            report(lv.right, lv.rightPtr[lpos], y2, out);   // canonical subtree
            lpos = lv.leftPtr[lpos];
            lv = lv.left;
        } else {
            lpos = lv.rightPtr[lpos];
            lv = lv.right;
        }
    }
    addLeafIfInside(lv.point, x1, x2, y1, y2, out);

    // PHASE 3: right spine follows x2; left children are canonical
    Node<T> rv = split.right;
    int rpos = split.rightPtr[posSplit];
    while (!rv.isLeaf) {
        if (x2 >= rv.splitX) {
            report(rv.left, rv.leftPtr[rpos], y2, out);     // canonical subtree
            rpos = rv.rightPtr[rpos];
            rv = rv.right;
        } else {
            rpos = rv.leftPtr[rpos];
            rv = rv.left;
        }
    }
    addLeafIfInside(rv.point, x1, x2, y1, y2, out);
    return out;
}

private void report(Node<T> w, int start, double y2, List<Point<T>> out) {
    Point<T>[] ys = w.ys;
    for (int p = start; p < ys.length && ys[p].y() <= y2; p++) {
        out.add(ys[p]);
    }
}
```

What the code does

- `pos` **is the core of the method.** At every node it holds "the index of the first element with y ≥ y1 in this node's `ys` ". Each move to a child updates it with `pos = v.leftPtr[pos]` or `pos = v.rightPtr[pos]` , which is **O(1)**. There is no binary search below the root.

- `report()` **never checks y ≥ y1**, because `start` already guarantees it. It only checks `y ≤ y2`, and it stops at the first failure since the array is sorted. The cost is O(1 + points reported).
- **Why a canonical subtree needs no x check (left spine):** everything under `split.left` has x ≤ `split.splitX` ≤ x2. If x1 ≤ `lv.splitX`, then everything in `lv.right` has x ≥ `lv.splitX` ≥ x1. So the whole subtree is inside `[x1,x2]`, and only y needs checking.
- **Leaves are checked on all four sides** by `addLeafIfInside`, because a spine can end on a point that is just outside the range.
- **Duplicate x-values** are handled by the choice of strict and non-strict comparisons: `x2 < splitX` to go left, `x1 > splitX` to go right, `x1 <= splitX` on the left spine, `x2 >= splitX` on the right spine. None of them can drop a point equal to `splitX`.

Worked trace: query `[5, 25] × [10, 26]` (real output)

PHASE 1 — FINDING THE SPLIT NODE

- `pos = lowerBoundY(root.ys, 10)` → **3** (`A2·12`, the first value ≥ 10). This is the only binary search in the query.
- At the root, `splitX = 13`. `x2=25 < 13`? No. `x1=5 > 13`? No. So the **root is the split node**, with `posSplit = 3`.

PHASE 2 — LEFT SPINE (FOLLOWS X1 = 5)

- `lv` = node(splitX=6), `lpos = root.leftPtr[3] = 1`
  - `x1=5 ≤ 6`, so **`lv.right` = node(10) is canonical.** Start at `node6.rightPtr[1] = 0`, which gives `[A3·22, A4·27]`.
    - A3·22 ≤ 26 → **report A3** ✓. A4·27 > 26 → stop.
  - Go left: `lpos = node6.leftPtr[1] = 1`, `lv` = node(3)
- `lv` = node(splitX=3): `x1=5 ≤ 3`? No, so go right: `lpos = rightPtr[1] = 0`, `lv` = leaf A2
- Leaf **A2 (6,12)** is inside → **report A2** ✓

PHASE 3 — RIGHT SPINE (FOLLOWS X2 = 25)

- `rv` = node(splitX=22), `rpos = root.rightPtr[3] = 2`
  - `x2=25 ≥ 22`, so **`rv.left` = node(20) is canonical.** Start at `node22.leftPtr[2] = 1`.
    - Its array is `[A7·6, A5·25]`. Starting at index 1 **skips A7·6 without any comparison.**
    - A5·25 ≤ 26 → **report A5** ✓
  - Go right: `rpos = rightPtr[2] = 1`, `rv` = node(26)
- `rv` = node(splitX=26): `25 ≥ 26`? No, so go left to leaf A8
- Leaf **A8 (26,9)** has x = 26 > 25 → not reported

Result: **`[A3, A2, A5]`.** A brute-force check agrees: A2 (6,12), A3 (10,22) and A5 (22,25) are exactly the ambulances inside that rectangle.

> **Where cascading paid off:** at node(20), a plain range tree would binary search `[6, 25]` for 10. Here the pointer jumped straight to index 1. Over a tree with n = 1,000,000 that is about 20 levels, so a plain range tree does 20 binary searches of about 20 steps each (~400), while this one does a single search of about 20 steps plus a pointer step per level.

Complexity

- There are O(log n) nodes on the spines, and each costs O(1) thanks to the pointers.
- There are O(log n) canonical nodes, each costing O(1) plus what it reports.
- **Total: O(log n + k).** The plain range tree is $O(\log^2 n + k)$.

Algorithm 5 — Nearest Neighbour (growing window)

Idea

A range tree answers rectangle queries, not "closest point" queries. So:

1. Query a small square around the incident, and keep **doubling** it until something matching the filter turns up.
2. Take the Euclidean distance `d` to the best hit.
3. **Query once more** with a square of half-side `d`, and return the true minimum.

Code

```
public Point<T> nearest(double qx, double qy, Predicate<Point<T>> keep) {
    if (root == null) return null;

    // Largest half-side ever needed to cover the whole data set
    double cover = Math.max(Math.max(qx - minX, maxX - qx),
                            Math.max(qy - minY, maxY - qy));
    if (cover < 0) cover = 0;

    double r = 1.0;                                  // start with a 1 km window
    List<Point<T>> hits = null;
    while (true) {
        hits = rangeQuery(Rectangle.around(qx, qy, r), keep);
        if (!hits.isEmpty() || r >= cover) break;
        r = Math.min(r * 2.0, cover);                // double, capped at cover
    }
    if (hits.isEmpty()) return null;                 // nothing matches anywhere

    double best = Double.POSITIVE_INFINITY;
    for (Point<T> p : hits) best = Math.min(best, p.distanceTo(qx, qy));

    // Exact pass: the true nearest lies within distance 'best'
    Point<T> winner = null;
    double winnerDist = Double.POSITIVE_INFINITY;
    for (Point<T> p : rangeQuery(Rectangle.around(qx, qy, best), keep)) {
        double d = p.distanceTo(qx, qy);
        if (d < winnerDist) { winnerDist = d; winner = p; }
    }
    return winner;
}
```

What the code does

- `cover` is the half-width of the smallest square around the incident that contains the whole bounding box. Capping `r` at `cover` **guarantees the loop ends**: once the window covers everything and still finds nothing, no point matches, and the method returns `null`.
- `keep` **is a predicate**, such as "is an available ambulance". The same tree answers "nearest ambulance", "nearest fire truck" and "nearest ICU hospital".
- **Why the first hit isn't the answer:** the window is a *square*. A point in its corner can be farther away than a point just outside its edge. For example, a hit at (+4, +4) is 5.66 km away, while a missed point at (+4.5, 0) is only 4.5 km away.
- **Why the second pass is always right:** any point closer than `d` has |dx| < d and |dy| < d, so it **must** be inside the square of half-side `d`. Taking the minimum over that square cannot miss the true nearest point.

Worked trace: accident at (18, 23) (real output)

```
cover = 19
window r=1  [17..19] x [22..24]   hits = []
window r=2  [16..20] x [21..25]   hits = []
window r=4  [14..22] x [19..27]   hits = [A5]
best candidate distance d = 4.4721        (sqrt(4^2 + 2^2) = sqrt(20))
exact pass box [13.53..22.47] x [18.53..27.47]   hits = [A5]
-> nearest = A5 at (22,25), 4.47 km
```

That is the same answer the demo prints. In the exact pass, A4 at (13,27) just misses because 13 < 13.53, so it cannot be closer.

Complexity

Each window is one O(log n + k) range query. The number of doublings depends on **map geometry** (about log(cover / 1 km)), not on n.

> **Honest detail:** the predicate filters **after** points are reported, so `k` counts every point in the window, including busy units the filter then discards.

Algorithm 6 — Radius Search

Code

```
public List<Point<T>> withinRadius(double qx, double qy, double r,
                                   Predicate<Point<T>> keep) {
    List<Point<T>> out = new ArrayList<Point<T>>();
    for (Point<T> p : rangeQuery(Rectangle.around(qx, qy, r))) {
        if (p.distanceTo(qx, qy) <= r && keep.test(p)) out.add(p);
    }
    return out;
}
```

What the code does

A circle isn't a rectangle, so this is a **filter-and-refine** approach:

1. **Filter:** run a range query on the circle's bounding square, `[qx−r, qx+r] × [qy−r, qy+r]`. That is O(log n + k).
2. **Refine:** keep only points whose Euclidean distance is ≤ r and that pass the predicate. `DispatchSystem` passes `ICU_HOSPITAL`, which is `type == HOSPITAL && icuBeds > 0`.

Demo: within 5 km of (7, 11) → **City-General, 3.16 km** ($\sqrt{(3^2 + 1^2)} = \sqrt{10}$).

Algorithm 7 — Point Location (slab decomposition)

Idea

Draw a vertical line through every left and right zone edge. Between two neighbouring lines is a **slab**. Inside a slab, the zones are just a stack of y-intervals that don't overlap. Finding a zone is then **two binary searches**: one on x to pick the slab, one on y to pick the interval.

Build

```
public PointLocation(List<Rectangle> rects, List<Z> labels) {
    // 1. Distinct, sorted x-boundaries -> slabs
    TreeSet<Double> xs = new TreeSet<Double>();
    for (Rectangle r : rects) { xs.add(r.minX()); xs.add(r.maxX()); }
    slabX = new double[xs.size()];
    int idx = 0;
    for (Double d : xs) slabX[idx++] = d;

    int slabs = Math.max(slabX.length - 1, 0);
    ...
    // 2. For each slab, collect the rectangles covering it, sorted by lower y
    for (int s = 0; s < slabs; s++) {
        double mid = (slabX[s] + slabX[s + 1]) / 2.0;
        List<Integer> cover = new ArrayList<Integer>();
        for (int i = 0; i < rects.size(); i++) {
            Rectangle r = rects.get(i);
            if (r.minX() <= mid && mid <= r.maxX()) cover.add(i);
        }
        cover.sort((a, b) -> Double.compare(rects.get(a).minY(),
                                            rects.get(b).minY()));
        ... // copy into loY[s], hiY[s], label[s]
    }
}
```

- The `TreeSet` removes duplicate boundaries and sorts them in one step.
- **Testing only the slab's midpoint is enough.** Every rectangle edge is a slab boundary, so a rectangle either spans the whole slab or doesn't touch its inside. Testing the midpoint avoids ambiguity at the boundaries.
- **Cost:** with n zones there are up to 2n−1 slabs, each scanning all n rectangles, so $O(n^2)$ space in the worst case.

Query

```
public Z locate(double px, double py) {
    if (slabX.length < 2) return null;
    if (px < slabX[0] || px > slabX[slabX.length - 1]) return null;  // off the map

    int s = upperBound(slabX, px) - 1;     // last boundary <= px
    if (s < 0) s = 0;
    if (s >= loY.length) s = loY.length - 1;  // px exactly on the right edge

    double[] lo = loY[s];
    double[] hi = hiY[s];
    int k = upperBound(lo, py) - 1;        // last interval starting <= py
    if (k >= 0 && py <= hi[k]) {
        return (Z) label[s][k];
    }
    return null;                            // in a gap between zones
}

private static int upperBound(double[] a, double key) {   // first index with a[i] > key
    int lo = 0, hi = a.length;
    while (lo < hi) {
        int mid = (lo + hi) >>> 1;
        if (a[mid] > key) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```

- `upperBound(...) - 1` gives "the last element ≤ key", which is the slab or interval the point starts in.
- **The clamp** handles a point exactly on the map's right edge (x = 30). There, `upperBound` returns 3, so `s` would be 2, which is out of range, and the clamp pulls it back to 1.
- `py <= hi[k]` catches points in a gap between zones.

Real structure and trace

```
slabX = [0, 15, 30]
  slab 0 (x 0-15):  loY=[0, 15]  hiY=[15, 30]  -> [South-West, North-West]
  slab 1 (x 15-30): loY=[0, 15]  hiY=[15, 30]  -> [South-East, North-East]
```

`locate(19, 24)`:

1. `upperBound([0,15,30], 19)` = 2, so **s = 1**
2. `upperBound([0,15], 24)` = 2, so **k = 1**
3. `24 ≤ hiY[1] = 30` ✓ → **North-East**

Real results for edge cases: `locate(15,15)` → North-East, `locate(30,30)` → North-East, `locate(100,100)` → `null`.

> **If asked "what about a point exactly on a border?":** the zones are closed rectangles, so both technically contain it. `upperBound` always gives the tie to the zone **above and to the right**. The rule is deterministic, so every call has exactly one owning sector.

Algorithm 8 — Allocation by Severity

Code

```
private static Predicate<Point<EmergencyResource>> availableType(ResourceType t) {
    return p -> p.payload().type() == t && p.payload().isAvailable();
}

public DispatchPlan allocate(Incident inc) {
    DispatchPlan plan = new DispatchPlan(inc);
    plan.setZone(zoneOf(inc.x(), inc.y()));                    // Algorithm 7

    int need = inc.severity().ambulancesNeeded();               // CRITICAL = 2
    for (int i = 0; i < need; i++) {
        EmergencyResource amb = nearestAvailable(inc.x(), inc.y(),
                ResourceType.AMBULANCE);                         // Algorithm 5
        if (amb == null) {
            plan.addUnmet("No ambulance available (" + (need - i) + " short)");
            break;
        }
        amb.setAvailable(false);            // hidden from later searches
        plan.addAmbulance(amb);
    }

    if (inc.severity().requiresIcu()) {
        EmergencyResource h = nearestIcuHospital(inc.x(), inc.y());
        if (h != null) { h.reserveIcuBed(); plan.setHospital(h); }
        else plan.addUnmet("No ICU hospital available");
    }
    if (inc.type().needsFireTruck()) { ... nearest FIRE_TRUCK, mark busy ... }
    if (inc.type().needsPolice())    { ... nearest POLICE_UNIT, mark busy ... }
    return plan;
}
```

What the code does

This is a **greedy** algorithm: each required unit goes to the nearest one still free.

- **The rules are data, not `if` chains.** `Severity.java` stores `(ambulances, requiresIcu)` for each level, and `IncidentType.java` stores `(needsFireTruck, needsPolice)`. Changing the dispatch policy only means editing an enum.
- **Lazy deletion is the key design choice.** `amb.setAvailable(false)` does **not** take the ambulance out of the tree. The predicate reads `isAvailable()` at query time, so the next `nearestAvailable` call skips it automatically. Dispatching costs **O(1) with no rebuild**.
- The same thing happens with hospitals. `reserveIcuBed()` decrements the bed count, and once it reaches 0, `hasIcu()` is false and `ICU_HOSPITAL` hides the hospital.
- **Shortages are recorded, not ignored.** The `unmet` list tells the dispatcher what couldn't be sent.

Worked trace: E1, a CRITICAL road accident at (19, 24) (matches the demo)

Step	Rule	Nearest match	Distance
Zone	point location	**North-East**	—
Ambulance #1	CRITICAL needs 2	**A5** (22,25)	√10 = 3.16 km
—	A5 is now busy and hidden		
Ambulance #2		**A4** (13,27)	√45 = 6.71 km
ICU hospital	CRITICAL requires ICU	**H2** Northside-Med (12,25)	√50 = 7.07 km
Fire truck	ROAD_ACCIDENT needs one	**F2** (24,22)	√29 = 5.39 km
Police	ROAD_ACCIDENT needs one	**P1** (8,18)	√157 = 12.53 km

H3 East-Care has 0 ICU beds, so the `ICU_HOSPITAL` predicate never returns it.

> **If asked "is greedy optimal?":** no, not across several incidents. Sending the nearest unit to call 1 can leave call 2 worse off. Optimal assignment of many units to many incidents is a matching problem (the Hungarian algorithm, $O(n^3)$). Greedy is the right choice for **one call at a time as it arrives**, because a dispatcher can't wait for future calls.

Algorithm 9 — Redistribution

Code

```
public List<String> redistributeCoverage() {
    List<String> log = new ArrayList<String>();
    for (Zone z : zones) {
        if (!availableAmbulancesInRegion(z.area()).isEmpty()) continue;  // still covered

        double cx = (z.area().minX() + z.area().maxX()) / 2.0;           // zone centre
        double cy = (z.area().minY() + z.area().maxY()) / 2.0;

        EmergencyResource donor = nearestSurplusAmbulance(z);
        if (donor == null) {
            log.add(z.name() + ": uncovered, no surplus ambulance to move");
            continue;
        }
        log.add(String.format(...));
        relocate(donor.id(), cx, cy);
    }
    return log;
}

private EmergencyResource nearestSurplusAmbulance(Zone needy) {
    ...
    for (EmergencyResource r : resources) {
        if (r.type() != ResourceType.AMBULANCE || !r.isAvailable()) continue;
        Zone home = zoneOf(r.x(), r.y());
        if (home == needy) continue;
        if (home != null && availableAmbulancesInRegion(home.area()).size() <= 1) {
            continue;   // don't leave the donor zone with none
```

```
        }
        double d = dist(r, cx, cy);
        if (d < bestD) { bestD = d; best = r; }
    }
    return best;
}

public void relocate(String resourceId, double newX, double newY) {
    ... // replace the resource with a new object at (newX, newY)
    rebuildIndex();                  // positions changed, so rebuild the tree
}
```

What the code does

1. **Find uncovered zones** with a range query per zone: a zone is uncovered if it has zero free ambulances.
2. **Pick a donor:** the free ambulance nearest the needy zone's centre, **excluding** any zone that would drop to zero. Without this guard the algorithm would just move the gap to another zone.
3. **Move it:** `x` and `y` are `final` in `EmergencyResource`, so relocating creates a new object. The **tree is then rebuilt in O(n log n)**. This is the only operation that triggers a rebuild, because it is the only one that changes a coordinate.

Worked trace (demo query 5)

Calls E4 at (20,6) and E5 at (26,9) use up **A7** and **A8**. **South-East** now has 0 ambulances, and its centre is (22.5, 7.5).

Candidate	Home zone (free units)	Distance to (22.5, 7.5)
A1 (3,4)	SW (2)	19.81
A2 (6,12)	SW (2)	17.10
A3 (10,22)	NW (2)	19.14
A4 (13,27)	NW (2)	21.69
A5 (22,25)	NE (2)	17.51
A6 (27,20)	NE (2)	**13.29** ✓
A7, A8	—	busy, skipped

→ **A6 moves to (22.5, 7.5)**. South-East's response distance goes from **13.29 km to 0.00 km**, and North-East keeps A5.

> **Honest complexity note:** `nearestSurplusAmbulance` is a **linear scan** over resources, and it runs a range query inside the loop. That is fine for a city's ambulance fleet, but it is not logarithmic. An improvement would be to keep a count of free ambulances per zone that updates on each dispatch or return.

Algorithm 10 — Coverage Map

```
for (Zone z : zones) {
    int ambulances = availableAmbulancesInRegion(z.area()).size();              // range query
    boolean icu = !hospitalsWithIcuWithinRadius(cx, cy, icuRadiusKm).isEmpty(); // radius query
    EmergencyResource nearest = nearestAvailableAmbulance(cx, cy);              // nearest neighbour
    String flag = (ambulances == 0) ? "  <-- UNDER-COVERED" : "";
    ...
}
```

This reuses Algorithms 4, 5 and 6 on each zone and builds a table from the results. With z zones, one report is O(z · (log n + k)). It is a good example of the structures being **reusable building blocks**.

Algorithm 11 — Checking Against Brute Force

```
private void rangeQueriesMatchBruteForce() {
    Random rng = new Random(7);                        // fixed seed, same run every time
    for (int t = 0; t < 300 && ok; t++) {
        List<Point<Integer>> pts = randomPoints(rng, 1 + rng.nextInt(120), 40);
        RangeTree2D<Integer> tree = new RangeTree2D<Integer>(pts);
        for (int q = 0; q < 20; q++) {
            Rectangle rect = randomRect(rng, 40);
            List<String> got = keys(tree.rangeQuery(rect));   // fast structure
            List<String> exp = bruteRange(pts, rect);         // O(n) loop
            Collections.sort(got);
            Collections.sort(exp);
            if (!got.equals(exp)) { ok = false; break; }
        }
    }
}
```

Why the testing holds up

- **A separate reference:** `bruteRange` is a plain loop with `if contains`. It is simple enough to be obviously correct, so it can judge the complex structure.
- **Fixed seeds** (7, 11, 13) make any failure reproducible.
- **Duplicates on purpose:** up to 120 random integer points on a 40 × 40 grid (1,600 cells) makes repeated coordinates almost certain. That exercises the tie-handling in `splitX` routing and in the cascade pointers.
- **Sorted comparison:** the tree and the loop report points in different orders, so both lists are sorted before comparing. This checks they hold exactly the same points.
- **Nearest neighbour compares distances, not identities** (with tolerance `1e-9`). Two points can be equally close, and returning either one is correct.

Scale: 300 trees × 20 queries = **6,000 range checks**, plus 3,000 nearest-neighbour and 2,000 radius checks. All pass.

Complexity Summary

Algorithm	Build	Query	Space
Range tree + fractional cascading	O(n log n)	**O(log n + k)**	O(n log n)
Plain range tree (for comparison)	O(n log n)	$O(\log^2 n + k)$	O(n log n)
Nearest neighbour	—	O(log n + k) per window	—
Radius search	—	O(log n + k)	—
Point location	$O(n^2 \log n)$ worst case	**O(log n)**	$O(n^2)$ worst case
Dispatch one unit	—	O(log n + k)	—
Mark busy / return to service	—	**O(1)**, no rebuild	—
Relocate one unit	O(n log n) rebuild	—	—
Naive scan (the baseline)	—	O(n)	O(n)

To explain the whole design in one sentence: **Algorithms 1–3 are preprocessing, Algorithm 4 is the fast search, 5–7 build other queries on top of it, 8–10 turn those queries into dispatch decisions, and 11 checks the fast search against a brute-force loop.**